

Voice-Based Communication Understanding

Analyzer Using Generative AI

Project Description:

The Voice-Based Communication Understanding Analyzer is a **Generative AI**-powered application that automatically evaluates how well a student or speaker understands a concept based purely on their **spoken explanation**. The user provides a written **Concept Reference** (the correct/expected explanation of a topic) and uploads the **Student Audio** recording (WAV/MP3) of the learner explaining that same concept in their own words. The platform transcribes the audio, uses a **Generative AI/LLM** model together with semantic similarity to compare the transcription against the reference concept, extracts objective audio features from the recording, and produces an overall **Understanding Score**, a qualitative grade (e.g., Poor, Fair, Good, Excellent), and AI-generated, human-readable feedback.

Built with **Streamlit** for the interactive web interface, **Librosa** for audio signal processing (waveform visualization, RMS Energy, Zero Crossing Rate, and Duration), a **Speech-to-Text engine** for transcription, and a **Generative AI model** combined with NLP **semantic similarity** for concept comparison and feedback generation, the Analyzer delivers an end-to-end communication assessment pipeline. Results — including the transcription, evaluation metrics, and AI-generated feedback — can be exported as a downloadable, professionally formatted **PDF Report** for record-keeping or sharing with instructors, trainers, or evaluators.

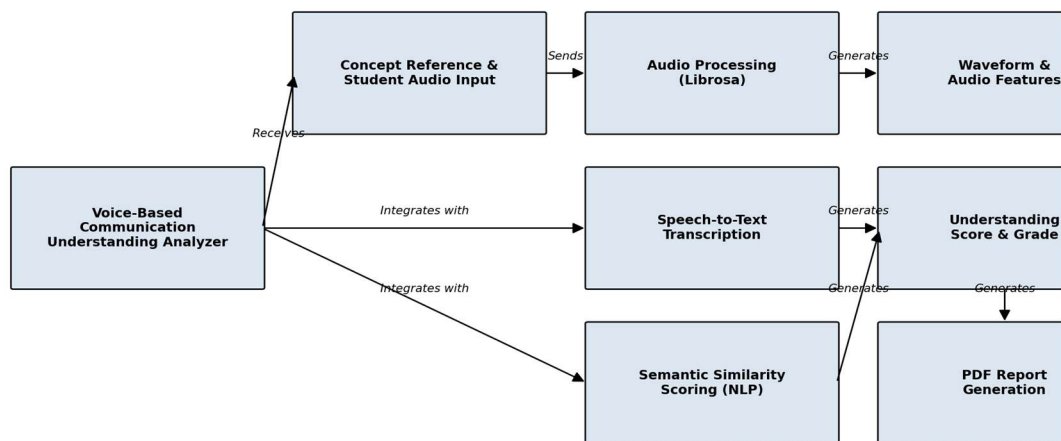
Scenarios

Scenario 1: A teacher uploads a reference explanation of “photosynthesis” and a student's recorded audio explaining the same concept in their own words. The Analyzer transcribes the recording, compares it against the reference, and returns a low semantic similarity score along with feedback noting the explanation differs significantly from the expected concept — flagging the student for additional review.

Scenario 2: A corporate trainer uses the Analyzer to verify that new hires can correctly articulate a company policy after a training session. Employees record a short verbal summary, and the tool generates an Understanding Score and a downloadable PDF report that the trainer can attach to the employee's onboarding file.

Scenario 3: A language-learning platform uses the Analyzer to assess a learner's spoken description of a grammar concept. The waveform and audio features (RMS Energy, Zero Crossing Rate, Duration) help instructors gauge speaking confidence and pace, while the semantic score measures conceptual accuracy, giving a holistic view of both delivery and comprehension.

Technical Architecture:



Description: The Analyzer uses a modular pipeline architecture built around a Streamlit frontend. The application receives two inputs — a written Concept Reference and an uploaded Student Audio file. The audio is processed with Librosa to render a waveform and extract quantitative features (Duration, Zero Crossing Rate, RMS Energy). In parallel, the audio is transcribed to text via a speech-to-text engine, and the resulting transcription is compared against the Concept Reference using a **Generative AI model** paired with a semantic similarity/NLP scorer. The Generative AI layer also drafts natural-language feedback explaining why the score was assigned. The combined outputs feed into a scoring and grading module that produces an Understanding Score, grade, and feedback, which is finally compiled into a downloadable PDF report.

(Note: If you are using a database in your project, definitely add an ER Diagram along with a description)

Pre-requisites:

Python: <https://www.python.org/downloads/>

FastAPI: <https://fastapi.tiangolo.com/>

Streamlit: <https://docs.streamlit.io/>

Librosa: <https://librosa.org/doc/latest/index.html>

Whisper: <https://github.com/openai/whisper>

Sentence-BERT: <https://www.sbert.net/docs/>

ReportLab: <https://www.reportlab.com/docs/reportlab-userguide.pdf>

Visual Studio Code: <https://code.visualstudio.com/>

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Generate a Generative AI API key and configure it securely in the project environment.
- **Activity 1.2:** Research and select an appropriate speech-to-text engine for transcribing spoken explanations.
- **Activity 1.3:** Select a Generative AI model and semantic similarity approach for concept comparison and feedback generation.
- **Activity 1.4:** Define the architecture of the application, detailing interactions between the audio input, transcription, AI scoring, and reporting modules.
- **Activity 1.5:** Set up the development environment, installing necessary libraries for Streamlit, Librosa, speech-to-text, and the Generative AI SDK.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the core functionalities: audio upload & waveform rendering, transcription, semantic similarity scoring, and audio feature extraction.
- **Activity 2.2:** Implement scoring/grading logic that converts similarity and audio metrics into an Understanding Score and qualitative grade.

Milestone 3: App.py Development

- **Activity 3.1:** Write the main application logic in app.py, wiring together the upload handler, analysis pipeline, and PDF generation.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the Streamlit interface for concept reference entry, audio upload, waveform display, and results presentation.
- **Activity 4.2:** Build dynamic result panels (Transcribed Explanation, Final Result, Audio Features, Evaluation Summary) that update after analysis.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring the server environment and ensuring all dependencies are correctly installed.
- **Activity 5.2:** Deploy and run the application using Streamlit, and verify functionality through the built-in Deploy/Rerun controls.

Milestone 6: Conclusion

Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the right generative AI model, speech-to-text engine, and semantic similarity components for evaluating spoken conceptual explanations, and on defining how the audio pipeline, NLP comparison, and reporting layers fit together.

Activity 1.1: Generate a Generative AI API Key

Before the application can request AI-generated feedback, you need a valid API key from your chosen Generative AI provider (e.g., Groq, OpenAI). Follow the steps below to create one and connect it securely to the project.

- **Step 1 — Create an Account:** Sign up for a free account with your chosen Generative AI provider using your Google account or email address.
- **Step 2 — Navigate to API Keys:** After logging in, open the API Keys section from your account dashboard.
- **Step 3 — Create a New API Key:** Click “Create API Key” and give it a descriptive name (e.g., voice-analyzer-key) so it is easy to identify later.
- **Step 4 — Copy and Store the Key:** Copy the generated key immediately and store it safely — most providers will not show it again.
- **Step 5 — Add the Key to Your Project:** Create a .env file in the project root and paste the key as shown below:

```
GENAI_API_KEY=your_copied_api_key_here
```

- **Step 6 — Verify the Key is Loaded:** The app.py file loads this key automatically at startup using python-dotenv:

```
from dotenv import load_dotenv
load_dotenv()
client = GenAIClient(api_key=os.environ.get("GENAI_API_KEY"))
```

- **Important — Never Commit Your API Key:** Add .env to your .gitignore file to prevent accidentally pushing the key to a public repository.

Activity 1.2: Select a Speech-to-Text Engine

- **Understand the Requirement:** The engine must accurately transcribe short (10–30 second) spoken explanations from uploaded WAV/MP3 files.
- **Evaluate Options:** Compare offline libraries (e.g., SpeechRecognition with a recognizer backend) against transformer-based models (e.g., Whisper) for accuracy and setup complexity.
- **Select the Engine:** Choose the transcription engine that best balances accuracy and response time for short educational audio clips.

Activity 1.3: Select a Generative AI Model and Semantic Similarity Approach

- **Explore NLP Options:** Review sentence-embedding models capable of comparing a reference concept sentence against a transcribed explanation.
- **Evaluate Performance:** Compare models on relevance to short conceptual text comparisons rather than long-document similarity.

- **Select the Model:** Choose a lightweight sentence-embedding or cosine-similarity approach that returns a normalized similarity score between 0 and 1.

Activity 1.4: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the pipeline: input capture, audio processing, transcription, similarity scoring, and PDF reporting.
- **Detail Frontend Functionality:** Outline how users enter the Concept Reference and upload Student Audio, and how waveform/results are displayed.
- **Outline Processing Responsibilities:** Specify how the backend extracts audio features, transcribes speech, computes similarity, and derives the final grade.
- **Describe Reporting Integration:** Define how the Understanding Score, RMS Energy, and feedback are compiled into a PDF report saved to disk and offered for download.

Activity 1.5: Set Up the Development Environment

- **Install Python:** Ensure Python 3.8+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```
D:\projects>python -m venv myenv
D:\projects>"d:\projects\venv\Scripts\activate"
(myenv) D:\projects> pip install -r requirements.txt
```

- **Install Streamlit:** Use pip to install Streamlit:

```
(myenv) D:\projects> pip install streamlit
```

- **Install Librosa:** Install Librosa for audio processing and feature extraction:

```
(myenv) D:\projects> pip install librosa
```

- **Install Speech-to-Text & NLP libraries:** Install the transcription and similarity libraries:

```
(myenv) D:\projects> pip install SpeechRecognition
(myenv) D:\projects> pip install sentence-transformers
```

- **Install PDF Generation Library:** Install a PDF library for report generation:

```
(myenv) D:\projects> pip install fpdf2
```

- **Set Up Application Structure:** Create the project directory structure including folders for reports, and main application files (app.py, requirements.txt).

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Analysis Features

Implement the core analysis features — audio handling, transcription, similarity scoring, and feature extraction — as a unified analysis pipeline triggered by a single button click.

- **Audio Uploader & Waveform:** Accepts a WAV/MP3 file, plays it back in-browser, and renders an amplitude-vs-time waveform plot.
- **Transcription Module:** Converts the uploaded audio into text using the selected speech-to-text engine.
- **Semantic Similarity Module:** Compares the transcribed text against the Concept Reference and returns a normalized similarity score.
- **Audio Feature Extractor:** Computes Duration, Zero Crossing Rate, and RMS Energy from the raw audio signal using Librosa.

Activity 2.2: Implement the Scoring and Grading Logic

Define Core Logic:

- Combine the semantic similarity score with audio-derived signals to compute an overall Understanding Score (0–100).
- Map score ranges to qualitative grades (e.g., Poor, Fair, Good, Excellent) with matching feedback text.

Process User Input:

- Validate that both a Concept Reference and a Student Audio file are provided before enabling analysis.
- Trigger the pipeline only when the user clicks “Analyze Concept Understanding.”
- Display a processing indicator (“Processing Audio...”) while transcription and scoring run.

Milestone 3: App.py Development

Milestone 3 focuses on creating and refining the core logic of the app.py file, which serves as the backbone of the Analyzer application. In this milestone, you'll wire together audio handling, transcription, scoring, and reporting into a single Streamlit script.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Application Flow:

- Render the page title, tagline, and layout using Streamlit's page configuration.
- Capture the Concept Reference via a text input/area widget.
- Capture the Student Audio via st.file_uploader, restricted to WAV/MP3 types.

```
concept_reference = st.text_area("Concept Reference")
audio_file = st.file_uploader("Upload Student Audio (WAV/MP3)", type=["wav", "mp3"])

if audio_file:
    st.audio(audio_file)
    plot_waveform(audio_file)
```

Analyze Concept Understanding button handler:

```
if st.button("Analyze Concept Understanding"):
    with st.spinner("Processing Audio..."):
        transcription = transcribe_audio(audio_file)
        similarity = compute_semantic_similarity(concept_reference, transcription)
        features = extract_audio_features(audio_file)
        score, grade, feedback = grade_understanding(similarity, features)

    st.subheader("Transcribed Explanation")
    st.write(transcription)
    st.subheader("Final Result")
    st.metric("Understanding", grade)
```

Generate PDF Report button handler:

```
if st.button("Generate PDF Report"):
    pdf_path = build_pdf_report(
        concept_reference, transcription, score, features, feedback
    )
    st.success(f"Saved to: {pdf_path}")
    st.write(f"PDF Path: {pdf_path}")
    st.write(f"File Exists: {os.path.exists(pdf_path)}")
    with open(pdf_path, "rb") as f:
        st.download_button("Download PDF Report", f,
            file_name="Communication_Report.pdf")
```

Set Up Handlers for Each Feature:

- For the analysis handler, read the Concept Reference and uploaded audio, then run transcription, similarity scoring, and feature extraction in sequence.
- Apply input validation: disable analysis and show a warning if either the Concept Reference or the audio file is missing.
- Route the “Generate PDF Report” action to a dedicated function that compiles all results into a saved PDF file and exposes it via a download button.

Define Helper Functions:

- **extract_audio_features()**: uses Librosa to load the audio and compute Duration, Zero Crossing Rate, and RMS Energy, returning them as a structured dictionary.

```
def extract_audio_features(audio_file):
    y, sr = librosa.load(audio_file)
    duration = librosa.get_duration(y=y, sr=sr)
    zcr = librosa.feature.zero_crossing_rate(y).mean()
    rms = librosa.feature.rms(y=y).mean()
    return {"Duration": duration, "Zero Crossing Rate": zcr, "RMS Energy": rms}
```

- **grade_understanding()**: maps the semantic similarity score to an Understanding Score out of 100 and a qualitative grade, then calls the Generative AI model to draft natural-language feedback.

```
def grade_understanding(concept_reference, transcription, similarity, features):
    score = round(similarity * 100, 2)
    grade = "Poor" if score < 40 else "Fair" if score < 70 else "Good"

    prompt = f"""Compare the student explanation to the reference concept and
    give one short, constructive feedback sentence.
```

```
Reference: {concept_reference}
Student Explanation: {transcription}
Understanding Score: {score}/100 ({grade})""

response = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[{"role": "user", "content": prompt}],
)
feedback = response.choices[0].message.content
return score, grade, feedback
```

Milestone 4: Frontend Development

Milestone 4 focuses on developing a clear, single-page Streamlit interface for the Analyzer. This involves laying out the concept/audio inputs, the waveform visualization, and the dynamically rendered results and report sections.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base Layout:

- Develop the main app.py page as a single-page interface with a header (title + tagline: “Automated evaluation of spoken conceptual explanations using AI.”).
- Arrange the Concept Reference text area and Student Audio uploader side-by-side or stacked at the top of the page.
- Display an inline audio player and waveform chart directly below the uploader once a file is provided.

Design the Results Layout:

- Use tabs or columns to present “Transcribed Explanation” and “Final Result” side-by-side after analysis.
- Show the grade (e.g., “Poor Understanding”) prominently, alongside Semantic Similarity and Duration metrics.
- Render the raw Audio Features (Duration, Zero Crossing Rate, RMS Energy) as an expandable JSON/metrics panel.

Activity 4.2: Creating Dynamic Result Rendering

- Attach the “Analyze Concept Understanding” button to trigger the full pipeline, showing a spinner labeled “Processing Audio...” while it runs.
- After analysis completes, dynamically reveal the Transcribed Explanation, Final Result, and Audio Features sections.
- Attach the “Generate PDF Report” button to compile the current results into a saved PDF and surface the save path, existence check, and a “Download PDF Report” button.

Milestone 5: Deployment

In Milestone 5, the focus is on running the Analyzer application locally to verify correct operation using Streamlit's built-in development server and Deploy/Rerun controls.

Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

```
D:\projects>python -m venv myenv
D:\projects>"d:\projects\venv\Scripts\activate"
(myenv) D:\projects>pip install -r requirements.txt
```

Activity 5.2: Local Testing and Verification

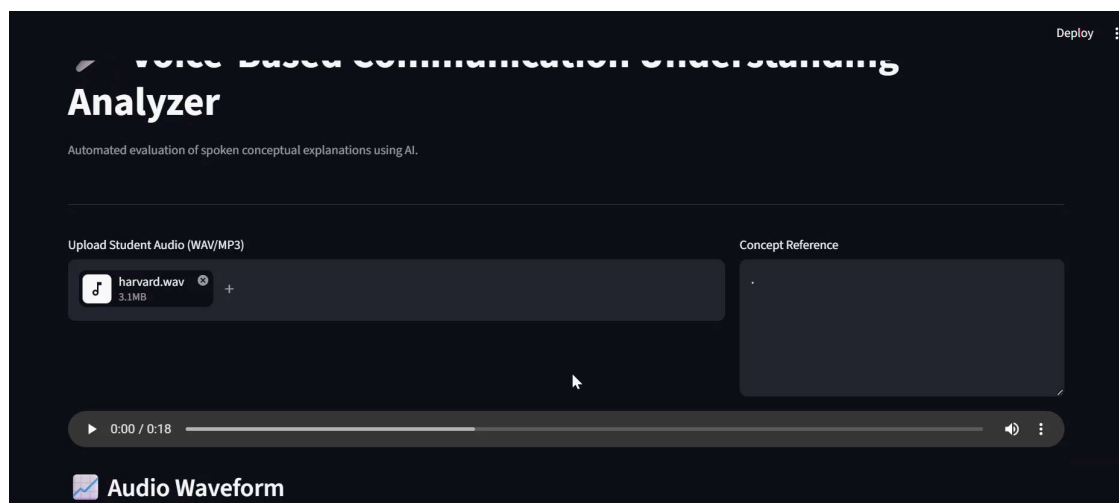
Start the Streamlit Development Server:

```
(myenv) D:\projects>streamlit run app.py
```

- Once running, Streamlit opens the app in the browser with the Deploy/Stop/Rerun toolbar visible in the top-right corner.
- Upload a sample audio file (e.g., harvard.wav) and enter a Concept Reference to verify the full pipeline: waveform rendering, transcription, scoring, and PDF report generation.
- Test the “Generate PDF Report” flow end-to-end and confirm the downloaded Communication_Report.pdf opens correctly and matches the on-screen results.

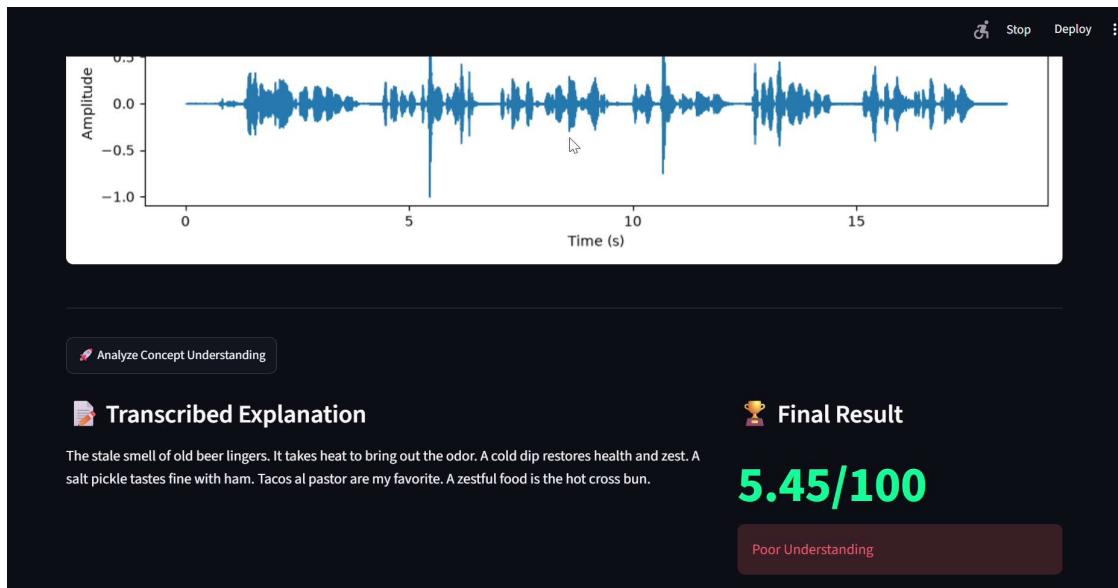
Exploring the Website's Web Pages:

Home / Analyzer Page:



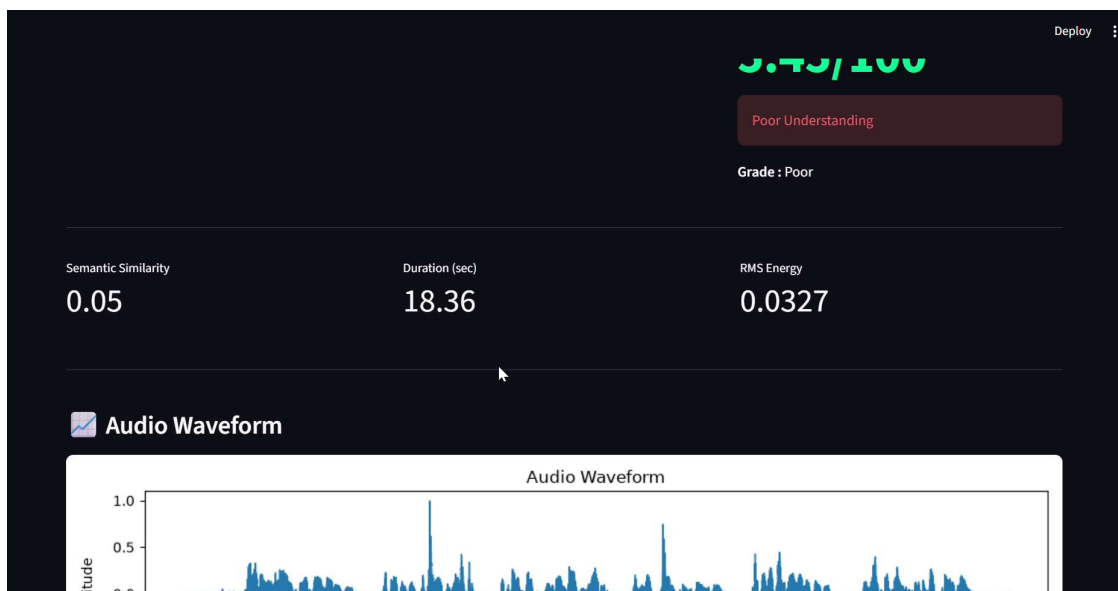
Description: The single-page interface of the Analyzer serves as both the landing page and the analysis tool. It displays the app title and tagline (“Automated evaluation of spoken conceptual explanations using AI.”), a Concept Reference input, and an audio uploader. Once a WAV/MP3 file is uploaded, an inline audio player and a live waveform chart appear on the right.

Results Section:



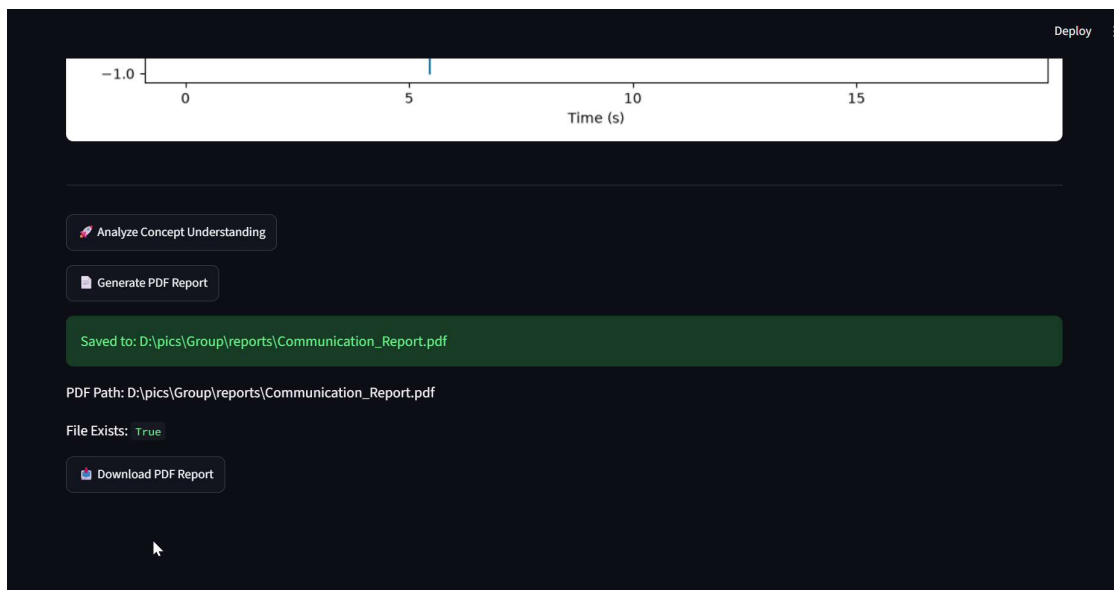
Description: After clicking “Analyze Concept Understanding,” the app displays the Transcribed Explanation and Final Result side-by-side, along with a prominent grade label (e.g., “Poor Understanding”). This section also surfaces Semantic Similarity and Duration metrics for quick review.

Audio Features Panel:



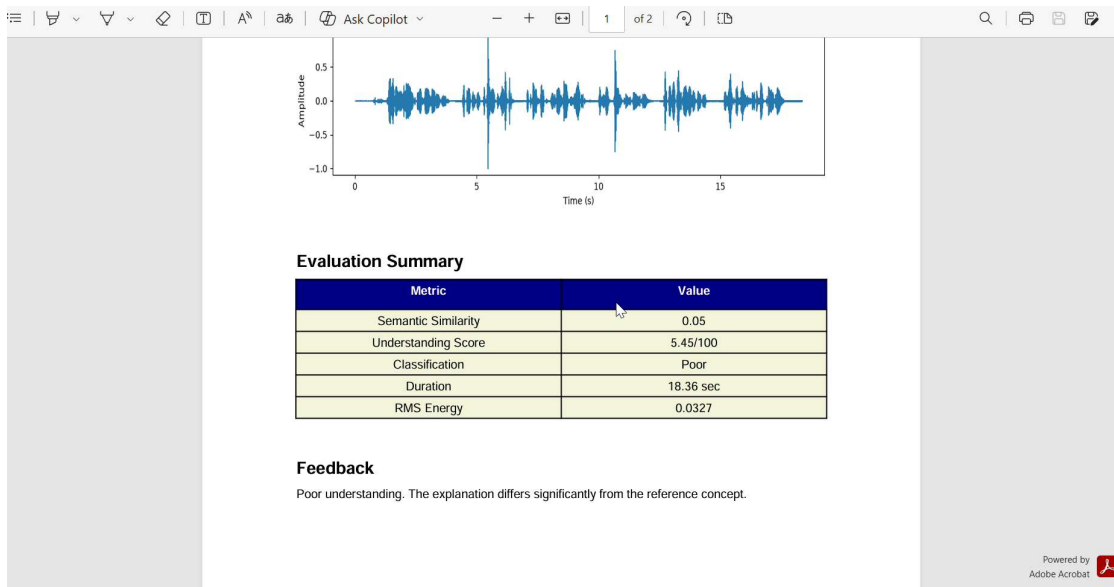
Description: An expandable panel shows the raw Audio Features extracted via Librosa — Duration, Zero Crossing Rate, and RMS Energy — presented as structured JSON for transparency, alongside the “Generate PDF Report” action.

PDF Report Generation & Download:



Description: Clicking “Generate PDF Report” saves a Communication_Report.pdf to disk and confirms the save path and file existence in the UI, with a “Download PDF Report” button provided to retrieve the file.

Generated PDF Report:



Description: The exported Communication_Report.pdf presents the Reference Concept and Student Transcription side-by-side, followed by the audio waveform and an Evaluation Summary table listing the Understanding Score, RMS Energy, and qualitative Feedback.

Conclusion:

The Voice-Based Communication Understanding Analyzer is a Generative AI-driven communication assessment platform built with Streamlit and powered by audio signal processing, speech-to-text

transcription, semantic similarity scoring, and an LLM-based feedback generator. It automatically evaluates how closely a spoken explanation matches a reference concept, combining objective audio metrics (Duration, Zero Crossing Rate, RMS Energy) with an Understanding Score, qualitative grade, and AI-generated feedback, then exports everything into a shareable PDF report. The project, which included generative AI model selection, pipeline development, and local deployment, demonstrates how targeted Generative AI and audio analysis can support educators, trainers, and language-learning platforms in assessing verbal comprehension at scale. Looking ahead, the Analyzer offers significant opportunities for expansion, such as multi-language transcription support, batch processing of classroom submissions, and richer rubric-based scoring. By continuously refining the underlying generative models and reporting templates, the platform is well-positioned to evolve into a comprehensive spoken-communication assessment assistant for schools and organizations of all sizes.